EXPERIMENT-67(HDOC)

C Programming

Program: Decimal to Binary Conversion Using Recursion

Step 1: Understand the Problem Statement

- Input: A non-negative decimal integer n. A negative number or a non-numeric value is rejected with an error message.
- Output: The binary representation of the number, displayed as `Binary of n = result`.
- The binary form of a number is written using only the digits 0 and 1. It is found with the recursive function `decimalToBinary(int n)`, which prints the bits directly.
- The number is repeatedly divided by 2 and the remainders (`n % 2`) are collected. Since the first remainder is the lowest bit, the remainders must be read in *reverse* order. Recursion does this naturally: the call for `n / 2` finishes (printing the higher bits) before the current remainder is printed.
- Base case: when n < 2, print n (it is already 0 or 1). Recursive case: otherwise call `decimalToBinary(n / 2)` first and then print `n % 2`. The number is halved at every call, so the recursion always ends. Zero is a valid input and gives 0.

Rule(s) used: decimalToBinary(n) prints n when n < 2; decimalToBinary(n) = decimalToBinary(n / 2) followed by (n mod 2) when n >= 2. Example: 10 → (10/2 = 5) → (5/2 = 2) → (2/2 = 1) → prints 1, then 0, then 1, then 0 = 1010.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Decimal number: 10`	Binary of 10 = 1010	Binary of 10 = 1010	Pass
2	`Decimal number: 25`	Binary of 25 = 11001	Binary of 25 = 11001	Pass
3	`Decimal number: 1`	Binary of 1 = 1	Binary of 1 = 1	Pass
4	`Decimal number: 0`	Binary of 0 = 0	Binary of 0 = 0	Pass
5	`Decimal number: 255`	Binary of 255 = 11111111	Binary of 255 = 11111111	Pass
6	`Decimal number: 1024`	Binary of 1024 = 10000000000	Binary of 1024 = 10000000000	Pass
7	`Decimal number: -5`	Invalid input. Enter a non-negative integer only.	Invalid input. Enter a non-negative integer only.	Pass
8	`Decimal number: abc`	Invalid input. Enter a non-negative integer only.	Invalid input. Enter a non-negative integer only.	Pass

Step 3: Algorithm

1. Start the program.
2. Read the decimal number n; if it is not a non-negative integer, display an error message and stop.

3. Call decimalToBinary(n).
4. In decimalToBinary(): if n < 2, print n and return (base case).
5. Otherwise call decimalToBinary(n / 2) (recursive case): this prints all the higher bits.
6. After the recursive call returns, print n % 2 (the current lowest bit).
7. When the recursion finishes, print a newline so the result is shown as Binary of n = result.
8. Stop the program.

Evaluation against Step 2 test cases: for 10 the calls are decimalToBinary(10) → (5) → (2) → (1); the base case prints 1, and on returning the remainders 0, 1, 0 are printed, giving 1010. For 25 the calls go 25 → 12 → 6 → 3 → 1, and the remainders 1, 0, 0, 1 follow the leading 1, giving 11001. For 1 and 0 the base case prints the number itself immediately. For 255 (eight 1s) every division leaves remainder 1, giving 11111111. For 1024 (2^10) every remainder is 0 until the final 1, giving a 1 followed by ten 0s. A negative number and a non-numeric entry are rejected as invalid input — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```c
/* HDOC Day-68: Convert a decimal number into its binary representation
   using RECURSION.
   Idea: repeatedly divide the number by 2. The remainders, read in
   reverse order, give the binary digits.
       decimalToBinary(n) = decimalToBinary(n / 2) followed by (n % 2)
   Base case     : n < 2  -> print n itself (0 or 1)
   Recursive case: n >= 2 -> convert n / 2 first, then print n % 2
   n is halved at every call, so the recursion always reaches the base case.
*/
#include <stdio.h>

/* Function prototype */
void decimalToBinary(int n);

int main() {
    int n;

    printf("Enter a decimal number: ");
    if (scanf("%d", &n) != 1 || n < 0) {
        printf("Invalid input. Enter a non-negative integer only.\n");
        return 1;
    }

    printf("Binary of %d = ", n);
    decimalToBinary(n);
    printf("\n");
    return 0;
}

/* Prints the binary representation of n recursively */
void decimalToBinary(int n) {
    if (n < 2) {                /* base case: 0 or 1 is already binary */
        printf("%d", n);
        return;
    }
    decimalToBinary(n / 2);     /* recursive case: higher bits first */
    printf("%d", n % 2);        /* then the current (lowest) bit */
}
```

Step 5: Compile, Execute and Evaluate

The program was compiled using the cc compiler and executed from the Linux terminal using the following commands:

`cc hdoc68.c` (compile)
`./a.out` (execute)

All 8 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

```
Session   Actions   Edit   View   Help

┌──(kali@kali)-[~]
└─$ nano hdoc68.c
┌──(kali@kali)-[~]
└─$ cc hdoc68.c
┌──(kali@kali)-[~]
└─$ ./a.out
Enter a decimal number: 10
Binary of 10 = 1010
┌──(kali@kali)-[~]
└─$ ./a.out
Enter a decimal number: 25
Binary of 25 = 11001
┌──(kali@kali)-[~]
└─$ ./a.out
Enter a decimal number: 1
Binary of 1 = 1
┌──(kali@kali)-[~]
└─$ ./a.out
Enter a decimal number: 0
Binary of 0 = 0
┌──(kali@kali)-[~]
└─$ ./a.out
Enter a decimal number: 255
Binary of 255 = 11111111
┌──(kali@kali)-[~]
└─$ ./a.out
Enter a decimal number: 1024
Binary of 1024 = 10000000000
┌──(kali@kali)-[~]
└─$ ./a.out
Enter a decimal number: -5
Invalid input. Enter a non-negative integer only.
┌──(kali@kali)-[~]
└─$ ./a.out
Enter a decimal number: abc
Invalid input. Enter a non-negative integer only.
┌──(kali@kali)-[~]
└─$
```